# Task 5: Apache Web Server, PHP and MariaDB – Dynamic Website Hosting

## Objective

Install the following packages and host a sample dynamic website with the help of a database.

Install Apache Web Server

Install PHP

Install MySQL/MariaDB

Configuration files, extensions, and versions should be noticed.

## Requirements

• AWS EC2 instance running RHEL

• Apache (httpd) web server

• PHP and PHP-FPM

• MariaDB database server

• PHP MySQL/MariaDB extensions

• Dynamic PHP webpage

• phpMyAdmin

• AWS Security Group configuration

• Windows client machine for browser testing

## Procedure

## Step 1 – Launch the EC2 Instance

An AWS EC2 instance running RHEL was used as the server for hosting the Apache web server, PHP application, and MariaDB database.

## Step 2 – Install Apache

The Apache web server package was installed on the RHEL EC2 instance using the dnf package manager.

sudo dnf install httpd -y

After installation, the Apache service was enabled so that it starts automatically on system boot.

sudo systemctl enable --now httpd

The status of the service was verified using:

sudo systemctl status httpd

The Apache service was confirmed to be active and running.

## Step 3 – Install PHP and PHP-FPM

PHP, PHP CLI, and PHP-FPM were installed on the RHEL EC2 instance.

sudo dnf install php php-cli php-fpm -y

The PHP-FPM service was enabled and started:

sudo systemctl enable --now php-fpm

The installed PHP version was verified using:

php --version

The installed PHP version was recorded as PHP 8.3.3.

## Step 4 – Configure PHP-FPM with Apache

Apache was configured to forward PHP requests to PHP-FPM.

The PHP configuration file used by Apache was:

/etc/httpd/conf.d/php.conf

The PHP-FPM handler configuration was verified:

<FilesMatch .(php|phar)$>
SetHandler "proxy:unix:/run/php-fpm/www.sock|fcgi://localhost"
</FilesMatch>

The PHP-FPM socket was identified as:

/run/php-fpm/www.sock

The Apache configuration was tested using:

sudo httpd -t

The configuration test returned "Syntax OK", confirming that the Apache configuration contained no errors.

Apache was restarted to apply the configuration:

sudo systemctl restart httpd

## Step 5 – Create and Test PHP Information Page

A PHP information page was created inside Apache's default document root:

sudo vi /var/www/html/info.php

The following PHP code was added:

<?php phpinfo(); ?>

The page was accessed using the EC2 instance's public IP address:

http://<EC2-Public-IP>/info.php

The PHP information page was displayed successfully, confirming that Apache and PHP-FPM were processing PHP requests correctly.

## Step 6 – Install MariaDB

The MariaDB database server package was installed using the dnf package manager.

sudo dnf install mariadb-server

The MariaDB service was enabled and started:

sudo systemctl enable --now mariadb

The installed MariaDB version was checked using:

mariadb --version

The installed version was recorded as MariaDB 10.11.x.

## Step 7 – Create the Database and Table

MariaDB was accessed using:

sudo mariadb

A database named website_db was created:

CREATE DATABASE website_db;

The database was selected:

USE website_db;

A users table was created to store user information:

```
CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);
```

The table was verified using:

SHOW TABLES;

The users table was successfully created.

## Step 8 – Create a Database User

A separate MariaDB user was created for the PHP website:

CREATE USER 'website_user'@'localhost' IDENTIFIED BY 'website@123';

The required privileges were granted on the website database:

GRANT ALL PRIVILEGES ON website_db.* TO 'website_user'@'localhost';

The granted privileges were verified using:

SHOW GRANTS FOR 'website_user'@'localhost';

The database user was then tested by logging in:

mariadb -u website_user -p

The login was successful.

## Step 9 – Install PHP Database Extensions

The PHP MySQL/MariaDB extension package was installed:

sudo dnf install php-mysqlnd

The installed PHP database extensions were verified using:

php -m | grep -E 'mysqli|pdo_mysql'

The following extensions were available:

mysqli

pdo_mysql

PHP-FPM was restarted after installing the extensions:

sudo systemctl restart php-fpm

These extensions allow PHP applications to communicate with the MariaDB database.

## Step 10 – Create the Dynamic PHP Website

A dynamic PHP website was created inside Apache's default document root:

sudo vi /var/www/html/index.php

The PHP application was configured to connect to the MariaDB database using the following database details:

Host: localhost

Username: website_user

Database: website_db

The PHP application used the mysqli extension to establish a connection with MariaDB.

A form was created to accept user information such as:

Name

Email

The submitted information was inserted into the users table in the MariaDB database.

The application also retrieved the stored records from the database and displayed them on the webpage.

The website was accessed using the EC2 instance's public IP address:

http://<EC2-Public-IP>/

The dynamic webpage was displayed successfully.

## Step 11 – Verify Database Connectivity

The MariaDB database was accessed using the dedicated website user:

mariadb -u website_user -p

The database was selected:

USE website_db;

The records stored in the users table were verified using:

SELECT * FROM users;

The data submitted through the PHP website was successfully stored in the MariaDB database and displayed using the SELECT query.

This confirmed that the PHP application was successfully connected to the MariaDB database.

## Step 12 – Install phpMyAdmin

The wget package was installed to download phpMyAdmin:

sudo dnf install wget

The phpMyAdmin package was downloaded:

wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz

The downloaded archive was extracted using:

tar -xzf phpMyAdmin-latest-all-languages.tar.gz

The extracted phpMyAdmin directory was moved into Apache's document root:

sudo mv phpMyAdmin-5.2.3-all-languages /var/www/html/phpmyadmin

The SELinux security context was restored:

sudo restorecon -Rv /var/www/html/phpmyadmin

This allowed Apache to access the phpMyAdmin files correctly.

## Step 13 – Access phpMyAdmin

phpMyAdmin was accessed through the web browser using the EC2 instance's public IP address:

http://<EC2-Public-IP>/phpmyadmin/

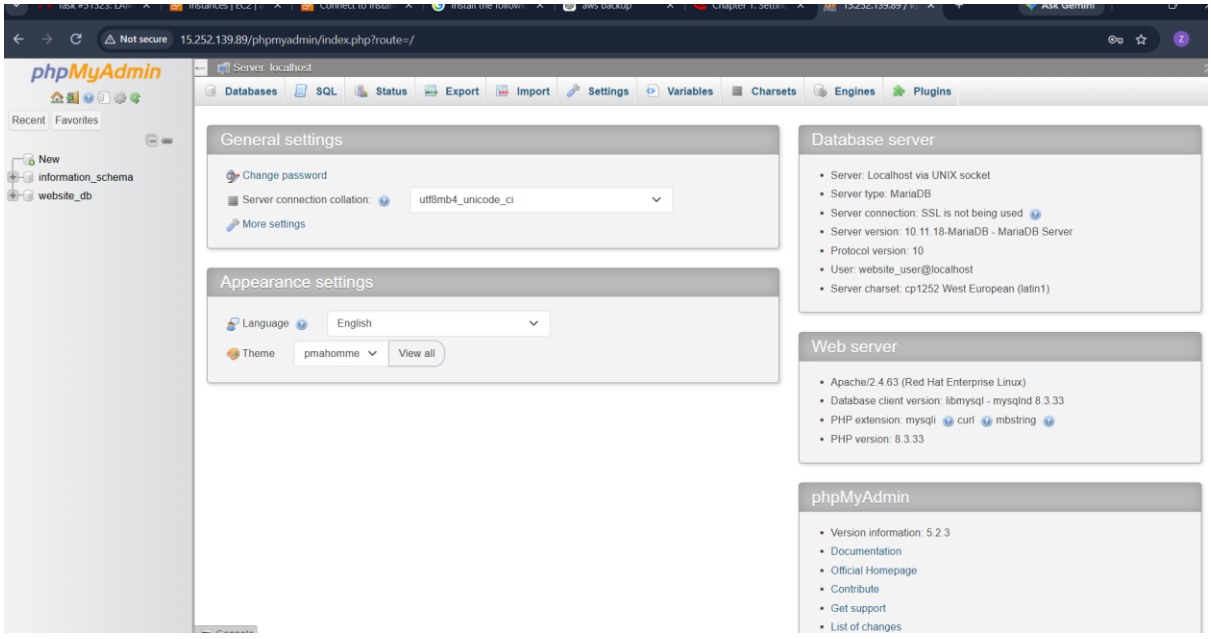
The MariaDB database user was used to log in:

Username: website_user

Password: website@123

The phpMyAdmin interface was displayed successfully.

The website_db database and users table were visible through the phpMyAdmin interface.



## Step 14 – Study Configuration Files, Extensions and Versions

The important configuration files related to the web server, PHP, PHP-FPM, and MariaDB were identified.

Apache main configuration file:

/etc/httpd/conf/httpd.conf

Apache PHP configuration file:

/etc/httpd/conf.d/php.conf

PHP configuration file:

/etc/php.ini

PHP-FPM configuration files:

/etc/php-fpm.conf

/etc/php-fpm.d/www.conf

MariaDB configuration file:

/etc/my.cnf

Apache default document root:

/var/www/html

phpMyAdmin installation directory:

/var/www/html/phpmyadmin

The important PHP extensions were identified as:

mysqli

pdo_mysql

The software versions were recorded as:

RHEL: 10.2

PHP: 8.3.3

MariaDB: 10.11.x

phpMyAdmin: 5.2.3

## Verification

• The Apache package was successfully installed on the RHEL EC2 instance.

• The Apache service was started and enabled to run automatically on boot.

• PHP and PHP-FPM were successfully installed and configured.

• The PHP information page was successfully displayed through Apache.

• The MariaDB database server was successfully installed and started.

• The website_db database was successfully created.

• The users table was successfully created.

• A dedicated database user website_user was created with privileges on website_db.

• The mysqli and pdo_mysql PHP extensions were successfully installed and verified.

• A dynamic PHP website was successfully created.

• The PHP website successfully connected to the MariaDB database.

• User data submitted through the website was successfully stored in the MariaDB database.

• The stored database records were successfully retrieved and displayed on the website.

• phpMyAdmin was successfully installed and accessed through the web browser.

• The website_db database and users table were successfully accessed through phpMyAdmin.

• The important Apache, PHP, PHP-FPM, and MariaDB configuration files were identified.

• The installed software versions and required PHP extensions were recorded.

## Conclusion

Through this task, Apache Web Server, PHP, PHP-FPM, and MariaDB were successfully installed and configured on an AWS EC2 instance running RHEL. A dynamic PHP website was created and connected to a MariaDB database using the required PHP database extensions. A dedicated database user was created and granted privileges on the application database. The website was tested by inserting and retrieving user data from the database. phpMyAdmin was also installed and successfully used to access and manage the MariaDB database through a web interface. This task provided practical, hands-on experience with Apache web server configuration, PHP and PHP-FPM, MariaDB database administration, PHP database connectivity, phpMyAdmin, and dynamic website hosting.